

OpenCode 101 — Los fundamentos

Una guía divulgativa

JA Palazón Ferrando

2026-06-14

Tabla de contenidos

1	Parte 1 — Los fundamentos	1
1.1	Un asistente que vive en tu ordenador	1
1.2	Por qué importa cómo llamas a las cosas	2
1.3	El ritual de la primera vez	2
1.4	El arte de pedir	2
2	Parte 2 — El taller de documentos	3
2.1	Contenido externo	3
2.2	Investigar con OpenCode	3
2.3	Reestructurar documentos	3
2.4	Traducir y adaptar a distintas audiencias	3
2.5	Documentos estructurados	3
3	Parte 3 — Productividad y cierre	4
3.1	Comandos personalizados	4
3.2	Revisión asistida	4
3.3	Compartir y entregar	4
4	Cierre	4
5	Referencias	5

1. Parte 1 — Los fundamentos

1.1. Un asistente que vive en tu ordenador

Vamos a empezar por lo básico, porque esto cambia todo: [OpenCode](#) no es un chat más. Cuando usas ChatGPT o Gemini, estás constantemente copiando y pegando información de un lado a otro. Es como tener un ayudante que te grita instrucciones desde la ventana de al lado.

OpenCode funciona al revés. Trabaja directamente sobre los *archivos locales* de tu ordenador. No le pasas copias: él abre el documento, lo lee, lo modifica y lo guarda. La información no viaja a servidores externos —la consulta sí va a la IA, pero el archivo vive en tu máquina— y el resultado se escribe donde tiene que escribirse.

Esto no es solo para programadores. Si trabajas con textos —informes, artículos, guiones, documentación— OpenCode te sirve. No necesitas saber programar. Necesitas saber explicar qué quieres. Y cuanto mejor sea tu *prompt*, mejores resultados obtienes.

1.2. Por qué importa cómo llamas a las cosas

El sistema de archivos es el mapa de tu proyecto. Si el mapa está ordenado, la IA se mueve bien. Si está caótico, se pierde.

Piénsalo como una cocina. Si tienes los ingredientes ordenados y etiquetados, cocinas sin pensar. El sistema de archivos es tu despensa.

Luego está la *terminal*. No es bonita, pero es directa. Solo necesitas cuatro comandos:

- `cd` para ir a un sitio
- `pwd` para saber dónde estás
- `ls` para ver qué hay
- `mkdir` para crear una carpeta

Cuatro palabras. Como aprender lo básico en un idioma nuevo. Con eso sobrevives.

Y los nombres. Nada de “Mi informe final (definitivo) v3.docx”. Usa minúsculas, guiones y números: `informe-final-v3.md`. Es una cortesía hacia la máquina y hacia ti mismo.

El formato ideal es [Markdown](#) (`.md`). Es texto plano, ligero, no se corrompe, se abre en cualquier programa. Es la lengua franca entre humanos y máquinas.

1.3. El ritual de la primera vez

Instalar OpenCode es un rito de paso. En Windows necesitas [WSL](#), en macOS usas `brew`, en Linux un comando `curl`. Todos llevan al mismo sitio.

Una vez instalado, abres OpenCode desde el *directorio de tu proyecto*. Ese directorio es tu hogar. Ahí se guarda el historial, la configuración y los archivos. Cada proyecto, su espacio.

Y luego están los dos modos. OpenCode tiene **Plan** y **Build**. **Plan** es para pensar: preguntas, analizas, diseñas. No toca nada, no cambia nada. Es modo seguro. **Build** es para hacer: crear archivos, modificarlos, ejecutar cambios. Alternas con la tecla `Tab`. No es un capricho técnico: primero imaginas, luego construyes. Es la forma natural de cualquier trabajo creativo.

Imagina que quieres hacer un documento educativo sobre Darwin. En **Plan** diseñas la estructura. Cuando lo tienes claro, cambias a **Build** y lo generas. Luego vuelves a **Plan** para analizar el resultado. Y a **Build** para añadir un cuestionario. Sin miedo a equivocarte.

💡 El ciclo natural del trabajo creativo

`Plan` → `Build` → `Plan` → `Build`. Piensa, prueba, evalúa, ajusta. OpenCode hace explícito lo que ya haces de forma natural.

1.4. El arte de pedir

Un buen *prompt* tiene tres partes: **contexto** (quién eres y para qué), **tono** (divulgativo, técnico, formal), y **formato** (secciones, tipo de archivo). Cuanto más contexto des, mejor te entiende.

La `@` es una herramienta pequeña pero poderosa. Cuando escribes `@` delante del nombre de un archivo, OpenCode lo lee y lo incorpora a la conversación. Es como decir “mira esto antes de responderme”.

💡 La arroba mágica

Usa `@` delante del nombre de cualquier archivo de tu proyecto para que OpenCode lo lea al instante. Funciona con `.md`, `.qmd`, `.csv`, `.pdf`, `.png` y más.

Y el ciclo: **Plan, Build, Plan, Build**. Es lo que haces naturalmente: piensas, pruebas, ves qué tal ha quedado, ajustas. OpenCode lo hace explícito. Es como maquetar con papel antes de cortar la tela.

2. Parte 2 — El taller de documentos

2.1. Contenido externo

OpenCode acepta contenido de varias maneras: pegar texto directamente en el chat, arrastrar una imagen a la terminal para que lea el texto que contiene, o referenciar un PDF con @ si está dentro de tu proyecto.

Puedes pegar un texto y pedirle que mejore la redacción, que lo adapte a un tono formal, que lo traduzca al inglés o que lo reescriba para niños de 10 años. Un mismo texto puede generar múltiples versiones sin esfuerzo.

Si trabajas con PDFs, OpenCode te sugerirá convertirlos a Markdown, porque ese formato es más ligero y manipulable para la IA.

2.2. Investigar con OpenCode

OpenCode puede buscar en internet por ti. Esto es útil cuando necesitas datos actualizados que el modelo no conoce, quieres comparar varias fuentes sobre un tema, o tienes un documento propio y quieres complementarlo con información externa.

Puedes pedirle que busque información actual y resuma los hallazgos, que compare fuentes diferentes y te diga en qué coinciden y en qué difieren, o que enriquezca un documento propio añadiendo una sección con datos recién obtenidos. Todo sin salir del chat.

2.3. Reestructurar documentos

Una de las grandes ventajas de trabajar con OpenCode es que puedes pedir cambios estructurales sobre documentos existentes sin tener que copiar y pegar manualmente.

Puedes dividir un archivo largo en varios más pequeños, fusionar varios archivos en uno solo, reordenar secciones dentro de un mismo documento, o cambiar la estructura de un texto continuo a otro formato como preguntas frecuentes o tabla.

El proceso es siempre el mismo: describes lo que quieres en **Build** y OpenCode lo ejecuta. Pero antes de hacer cambios importantes, conviene pasar a **Plan** para pensar la estructura.

2.4. Traducir y adaptar a distintas audiencias

Un mismo texto puede servir para públicos muy diferentes. OpenCode te permite crear versiones sin reescribir desde cero: traducir a otros idiomas manteniendo el formato, cambiar el tono según la audiencia (formal, divulgativo, técnico, infantil), y ajustar la extensión (versión ejecutiva de un párrafo, versión detallada, versión completa).

Puedes crear tres versiones de un mismo documento —ejecutiva para directivos, divulgativa para público general, técnica para expertos— con una sola instrucción. O adaptar un texto empresarial para explicárselo a un niño de 8 años usando ejemplos cotidianos.

2.5. Documentos estructurados

Más allá del texto continuo, OpenCode puede crear elementos formales que dan calidad profesional a tus documentos: tablas comparativas, listas de verificación, glosarios de términos técnicos, cabeceras profesionales con metadatos (título, autor, fecha, versión, estado), y plantillas reutilizables para informes.

Una plantilla bien hecha te ahorra horas de trabajo repetitivo. La creas una vez con las secciones que necesitas y la reutilizas en cada proyecto.

3. Parte 3 — Productividad y cierre

3.1. Comandos personalizados

Un comando personalizado es un atajo que le enseñas a OpenCode para que ejecute una tarea con solo escribir `/`. Por ejemplo, podrías tener `/faq` para convertir cualquier documento en preguntas frecuentes, `/plantilla` para generar un informe con tu estructura habitual, o `/revisar` para que OpenCode revise el documento activo.

Los comandos se definen en archivos `.md` dentro del directorio `.opencode/commands/` de tu proyecto. Cada archivo tiene un nombre (`faq.md`, `plantilla.md`, `revisar.md`) que se convierte en el nombre del comando. La primera vez que OpenCode escriba en esta carpeta, te pedirá permiso, ya que es el directorio de configuración del proyecto.

Una vez creados, los usas escribiendo `/nombre-comando documento.md` en el chat. Puedes tener tantos comandos como quieras: `/resumir`, `/traducir`, `/glosario`, `/check...` cada uno para una tarea repetitiva que quieras automatizar.

3.2. Revisión asistida

Antes de entregar un documento, siempre viene bien que alguien lo revise. OpenCode puede actuar como segundo par de ojos: detectar errores ortográficos y gramaticales, señalar incoherencias entre secciones, valorar el tono y la claridad del texto, e identificar lagunas de información.

El flujo ideal es: en **Plan** pides una revisión sin riesgo, analizas las sugerencias, pasas a **Build** y aplicas las que te parecen bien, y vuelves a **Plan** para una segunda vuelta si hace falta.

Puedes pedir una revisión general, detectar contradicciones entre secciones, evaluar si el tono es adecuado para una audiencia concreta, o pasar un checklist de entrega para asegurarte de que el documento cumple con los requisitos mínimos antes de publicarlo.

3.3. Compartir y entregar

Para cerrar el ciclo, OpenCode ofrece dos herramientas: `/share` genera un enlace a la conversación para compartir con compañeros, y `/export` descarga la conversación completa como archivo Markdown.

El enlace de `/share` es útil para enseñar a un colega cómo conseguiste un resultado, pedir opinión sobre el proceso seguido, o guardar una referencia de una sesión importante. La exportación con `/export` te permite repasar lo que hiciste en cualquier editor.

Además, es buen momento para revisar los archivos generados durante el trabajo y organizarlos en carpetas. Puedes pedirle a OpenCode que revise tu carpeta y te recomiende una estructura para organizarlos.

4. Cierre

Con esto completas los 12 pasos del curso. Has aprendido a:

- Entender qué es OpenCode y cómo trabaja sobre archivos locales
- Usar la terminal y los comandos básicos del sistema
- Diferenciar **Plan** y **Build** como dos fases del trabajo creativo
- Escribir **prompts** eficaces con contexto, tono y formato
- Trabajar con contenido externo: texto pegado, imágenes, PDFs
- Investigar en internet desde OpenCode
- Reestructurar documentos: dividir, fusionar, reordenar
- Traducir y adaptar textos a distintas audiencias

- Crear tablas, glosarios, checklists y plantillas
- Automatizar tareas repetitivas con comandos personalizados
- Revisar documentos como un segundo par de ojos
- Compartir sesiones, exportar conversaciones y organizar archivos

El [curso completo](#) incluye además una [página de ideas y ejemplos](#) para empresa, academia, deportes, hogar y aficiones, así como preguntas frecuentes para resolver dudas habituales.

Ahora es momento de poner en marcha tus propias ideas. Empieza con proyectos pequeños y poco comprometidos antes de lanzarte a proyectos más ambiciosos. Los fundamentos están aquí para sostenerte cuando algo no funcione.

5. Referencias

- [OpenCode](#) — Sitio oficial del asistente: descarga, documentación y guías de uso.
- [Curso OpenCode 101](#) — Los tres bloques del curso con ejemplos prácticos, FAQ y una página de ideas por sector.
- [Guía de Markdown](#) — Referencia completa del formato de texto plano que OpenCode usa para crear y editar documentos.
- [WSL para Windows](#) — Entorno Linux necesario para instalar y ejecutar OpenCode en Windows.
- [Homebrew](#) — Gestor de paquetes para instalar OpenCode en macOS.
- [OpenCode en GitHub](#) — Repositorio oficial del proyecto, ideal para seguir novedades y reportar incidencias.